PROBLÈME · ANNALES SPÉ · 2 HEURES

Deux problèmes de communication numérique

1. Analyse d'un texte, dictionnaires et SQL

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Alice compresse un message, l'encode, le transmet ; Bob le décode puis le décompresse. On étudie la compression (codage arithmétique) puis le décodage (algorithme de Viterbi).

Avec le code de longueur fixe `'a'` = 00, `'b'` = 01, `'c'` = 10, la chaîne `s='abaabaca'` tient sur 16 bits. On peut coder `'a'` sur 1 bit et `'b'`, `'c'` sur 2 bits pour obtenir 11 bits.

1. Proposer une telle représentation décodable sans ambiguïté, avec code(`'a'`) < code(`'b'`) < code(`'c'`).

2. Écrire `nbCaracteres(c:str, s:str) -> int` (nombre d'occurrences de `c` dans `s`), linéaire en $n =$ `len(s)`.

3. Que renvoie `listeCaracteres('abaabaca')` ? Expliquer son principe.

```
def listeCaracteres(s:str):
    listeCar = []
    n = len(s)
    for i in range(n):
        c = s[i]
        if not(c in listeCar):
            listeCar.append(c)
    return listeCar
```

4. Complexité dans le pire cas en fonction de n et du nombre k de caractères distincts (le test `c in listeCar` est linéaire en la longueur de `listeCar`).

5. Que fait `analyseTexte` ? Valeur de `analyseTexte('babaaaabca')` ?

```
def analyseTexte(s:str):
    R = []
    l = listeCaracteres(s)
    for i in range(len(l)):
        c = l[i]
        R.append((c, nbCaracteres(c, s)))
    return R
```

6. Complexité de `analyseTexte` en fonction de n et k.

7. Réécrire `analyseTexte` avec un dictionnaire, en complexité linéaire en n, indépendante de

k, avec un seul parcours de la chaîne. Exemple : `analyseTexte('abracadabra')` renvoie `{'a':5, 'b':2, 'r':2, 'c':1, 'd':1}`.

Base de données : `caractere(idCar, symbole, typeCaractere, codeHTML)` ; `corpus(idLivre, titre, auteur, annee, nombreCaracteres, langue)` ; `occurrences(idCar, idLivre, nombreOccurrences)`

1. Requête SQL : liste sans doublon des auteurs de `corpus`.
2. Requête SQL : pour chaque caractère, le symbole et sa fréquence en `'Français'` (nombre total d'occurrences divisé par le nombre total de caractères des textes du corpus).

2. Codage arithmétique

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Codage arithmétique : fréquences a : 0,2, b : 0,1, c : 0,2, d : 0,4, e : 0,1, d'où les intervalles a : $[0;0{,}2[$, b : $[0{,}2;0{,}3[$, c : $[0{,}3;0{,}5[$, d : $[0{,}5;0{,}9[$, e : $[0{,}9;1[$. On part de $[0;1[$ et, à chaque caractère, on garde la portion correspondante du sous-intervalle courant. Exemple : `'dac'` donne $[0{,}5;0{,}9[$, puis $[0{,}50;0{,}58[$, puis $[0{,}524;0{,}540[$. `codeCar(car, g, d)` renvoie le sous-intervalle de $[g,d[$ associé à `car`.

1. Intervalle correspondant à `'bac'`.
2. Écrire `codage(s:str) -> (float, float)` qui renvoie l'intervalle final (g,d).

Décodage : on repère dans quelle succession d'intervalles se trouve x. Pour $x = 0{,}123$: `'a'`, puis `'d'` (sous-intervalle $[0{,}10;0{,}18[$), etc.

1. Caractère qui suit `'ad'` pour $x = 0{,}123$, avec le sous-intervalle utilisé.
2. Donner deux chaînes correspondant à 0,2. D'où vient l'ambiguïté ?

On ajoute un caractère de fin `'#'`, de plage non vide près de 0. `decodeCar(x, g, d)` renvoie le caractère dont la plage dans $[g,d[$ contient x.

1. Écrire `decodage(x:float) -> str` (caractère `'#'` compris).

3. Décodage par l'algorithme de Viterbi

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Viterbi. Bob observe `Obs` $= [\text{obs}_0, \dots, \text{obs}_{N-1}]$, symboles de $\Sigma = [\![0, K-1]\!]$. $E[i][j]$: probabilité d'observer i sachant que j a été émis, ici $E = [[0.7, 0.2, 0.3], [0.2, 0.7, 0.1], [0.1, 0.1, 0.6]]$. $P[i][j]$: probabilité d'émettre j à l'instant $t{+}1$ sachant que i a été émis à l'instant t, ici $P = [[0.3, 0.2, 0.5], [0.4, 0.4, 0.2], [0.2$

Graphe : un sommet $S_{i,j}$ par symbole i et par instant j ; arcs de chaque $S_{i,j}$ vers tous les $S_{k,j+1}$; une source σ reliée à la première couche et une cible τ reliée à la dernière. Poids : $\sigma \to S_{i,0}$:

$E_{\text{obs}_0,i}$; $S_{i,j} \to S_{k,j+1}$: $E_{\text{obs}_{j+1},k} P_{i,k}$; vers τ : 1. La probabilité d'un chemin est le produit des poids ; on cherche le chemin de probabilité maximale.

1. Nombre de sommets et d'arcs (sans σ, τ et leurs arcs).
2. Pour `Obs = [2,0]`, construire le graphe pondéré.
3. Nombre de chemins de σ à τ ; une exploration exhaustive est-elle envisageable ?
4. Écrire `maximumListe(liste) -> (float, int)` : maximum et plus petit indice où il est atteint.
5. Écrire `glouton(Obs, P, E, K, N) -> [int]`, qui suit à chaque étape l'arc de plus forte probabilité (on dispose de `initialiserGlouton`).
6. Complexité de l'approche gloutonne.
7. Chemin glouton sur la figure 4 (arcs $\sigma \to 0$: 0,6, $\sigma \to 1$: 0,4 ; $0 \to 0$: 0,5, $0 \to 1$: 0,1, $1 \to 0$: 0,9, $1 \to 1$: 0,01 ; arcs vers τ : 1). Optimalité ?
8. Ramener la recherche à un plus court chemin à poids positifs ; quel algorithme ?

$T_{i,0} = E_{\text{obs}_0,i}$ et, pour $j \geqslant 1$, $T_{i,j} = \max_k T_{k,j-1} P_{k,i} E_{\text{obs}_j,i}$; `argT` garde l'indice k réalisant le maximum (-1 pour $j = 0$). On dispose de `initialiserViterbi(E, Obs0, K, N)` qui crée `T` et `argT` (K lignes, N colonnes) avec la colonne 0 remplie.

1. Écrire `construireTableauViterbi(Obs, P, E, K, N)` (de bas en haut).
2. Avec les tableaux donnés (dernière colonne de `T` : $1{,}8 \cdot 10^{-5}$, $3{,}5 \cdot 10^{-6}$, $1{,}3 \cdot 10^{-5}$; `argT` lignes : $[-1, 2, 0, 0, 2, 1, 1, 0]$, $[-1, 2, 2, 2, 2, 1, 1, 0]$, $[-1, 2, 0, 0, 2, 1, 1, 0]$), donner la séquence la plus probable.
3. Complexités temporelle et spatiale de la programmation dynamique.